K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Name of Student : Shlok Mitesh Rawal
Roll No : B - 73
Batch : BS4

EXPERIMENT 3

Aim:

Implementation of Infix to Postfix Expression for real-world application.

Objective:

- To understand the concept of **stack data structure**.
- To implement stack operations for expression conversion.
- To convert an **infix expression into a prefix expression**.
- To understand the precedence and associativity of operators.
- To perform expression conversion using stack operations.

Theory:

An **infix expression** is an expression in which the operator is written between the operands, such as A+B*C.

A **prefix expression** is an expression in which the operator is written before the operands, such as +A*BC.

A stack is used during the conversion process to temporarily store operators and

parentheses. The conversion from infix to prefix is performed by reversing the infix
expression, interchanging the opening and closing parentheses, converting the resulting expression into postfix, and finally reversing the postfix expression.

Operator precedence determines the order in which operators are processed. In this program, ^ has the highest precedence, followed by *, /, %, and then +, -.

Algorithm:

Step 1: Start.

Step 2: Read the infix expression.

Step 3: Reverse the infix expression and interchange (and).

Step 4: Scan the expression from left to right.

Step 5: If an operand is encountered, add it to the postfix expression.

Step 6: If (is encountered, push it onto the stack.

Step 7: If) is encountered, pop operators until (is found, then discard (.

Step 8: If an operator is encountered, pop operators with higher or equal precedence and then push the current operator.

Step 9: Pop all remaining operators from the stack.

Step 10: Reverse the postfix expression to obtain the prefix expression.

Step 11: Display the prefix expression.

Step 12: Stop.

Program :

```
#include<stdio.h>
#include<conio.h>
```

```
char stack[50];
int top = -1;
void push(char ch)
{
 top++;
 stack[top] = ch;
}

 char pop()
{
 char ch;

 ch = stack[top];
 top--;

 return ch;
}

int priority(char ch)
{
 if(ch == '+' || ch == '-')
 return 1;

 if(ch == '*' || ch == '/' || ch == '%')
return 2;

 return 0;
}

void infix_to_postfix(char infix[], char postfix[])
{
 int i = 0, j = 0;
 char ch;

 while(infix[i] != '\0')
 {
 ch = infix[i];
```

```
if((ch >= 'A' && ch <= 'Z') ||
(ch >= 'a' && ch <= 'z') ||
(ch >= '0' && ch <= '9'))
{
postfix[j] = ch;
j++;
}
else if(ch == '(')
{
push(ch);
}
else if(ch == ')')
{
while(top != -1 && stack[top] != '(')
{
postfix[j] = pop();
j++;
}

pop();
}
else
{
// pop operators with higher or equal priority  while(top !=
-1 && priority(stack[top]) >= priority(ch))  {
postfix[j] = pop();
j++;
}

push(ch);
}

i++;
}

while(top != -1)
{
postfix[j] = pop();
```

```
 j++;
 }

 // end of string
 postfix[j] = '\0';
}

void main()
{
 char infix[50], postfix[50];

 clrscr();

 while(1)
 {
 printf("\nenter infix expression (type exit to exit): ");
scanf("%s", infix);

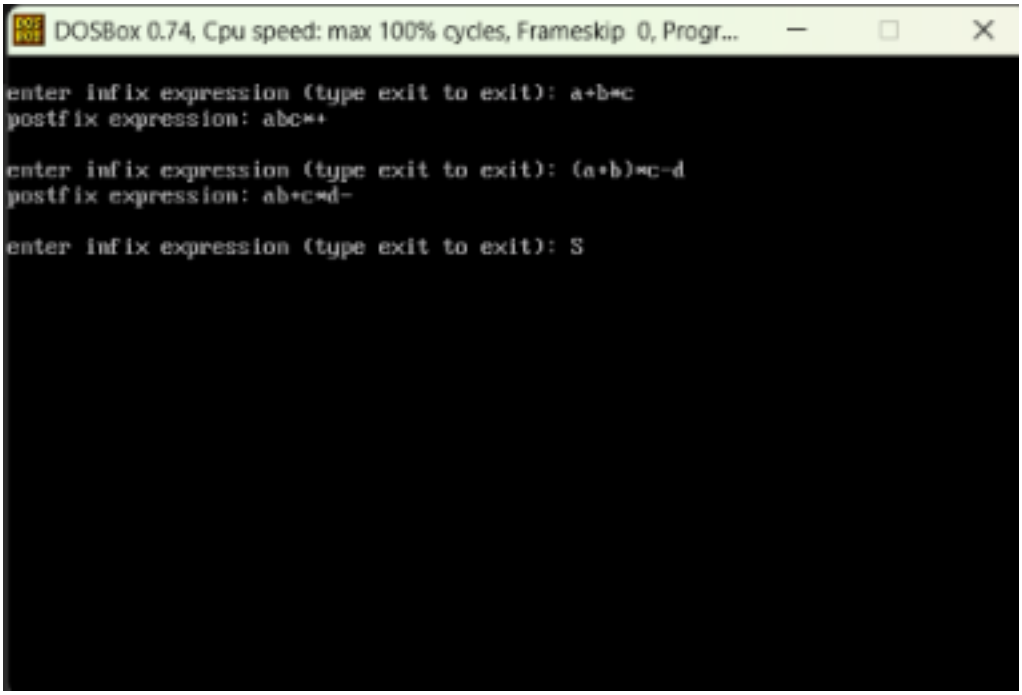
 if(infix[0] == 'e' &&
 infix[1] == 'x' &&
 infix[2] == 'i' &&
 infix[3] == 't' &&
 infix[4] == '\0')
 {
 printf("exited program\n");
 break;
 }

 infix_to_postfix(infix, postfix);

 printf("postfix expression: %s\n", postfix);  }

 getch();
}
```

Output :



Outcome :

The program successfully converts a given **infix expression into its corresponding prefix expression** using a stack and demonstrates the application of stack operations in expression conversion.

Conclusion :

Thus, the **infix expression was successfully converted into a prefix expression using stack operations**. The program demonstrates the use of stack, operator precedence, parentheses handling, and expression conversion in C programming.